

ARCHITECTURE TECHNIQUE

ChatDocs OHADA — v1.0 · Document de conception pour le développement

1. Vue d'ensemble

Les grands blocs du système et leurs responsabilités.



Principe directeur : le frontend ne contient aucune logique juridique et n'appelle jamais le LLM directement. Toute la chaîne de raisonnement, la clé d'API et le contrôle du quota vivent côté serveur. L'ingestion du corpus est un processus *hors ligne*, déclenché manuellement — jamais pendant qu'un utilisateur attend une réponse.

2. Pipeline d'ingestion (hors ligne)

Comment un PDF officiel devient un corpus interrogeable. Exécuté une fois par texte, puis à chaque mise à jour.

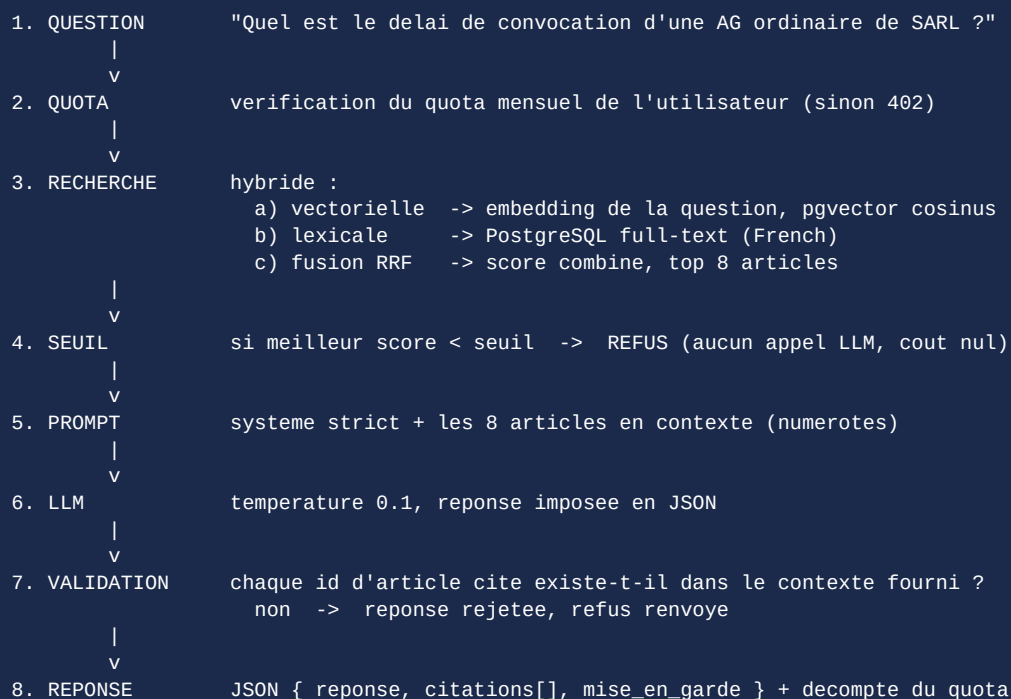
Étape	Traitement	Sortie
1. Collecte	Téléchargement du PDF officiel, calcul d'une empreinte SHA-256, archivage horodaté	Fichier source + fiche de provenance
2. Extraction	pdfplumber pour le texte natif ; Tesseract (OCR) si le PDF est scanné	Texte brut paginé
3. Découpage	Détection des en-têtes (LIVRE / TITRE / CHAPITRE / SECTION / Article N) par expressions régulières ; reconstitution du chemin hiérarchique	Liste d'articles avec chemin
4. Consolidation	Application des textes modificatifs ; clôture des anciennes versions (dateAbrogation) et création des nouvelles	Articles en vigueur + historique

Étape	Traitement	Sortie
5. Vérification	Contrôles automatiques (numérotation continue, articles vides, longueurs aberrantes) puis relecture humaine des articles modifiés	Rapport de contrôle validé
6. Embeddings	Vectorisation de chaque article (contenu + chemin hiérarchique en préfixe) ; insertion dans pgvector	Corpus interrogeable

Point d'attention : le préfixe hiérarchique injecté avant vectorisation (« CGI > Livre I > Titre II > Article 92 : ... ») améliore nettement la pertinence de la recherche, car un article isolé est souvent incompréhensible hors de son contexte.

3. Pipeline de requête (RAG)

Le trajet d'une question, de la saisie à la réponse sourcée.



Format de réponse imposé au modèle

```

{
  "reponse": "texte en francais, sans citation inventee",
  "citations": [ { "article_id": 4127, "extrait": "...", "pourquoi": "..." } ],
  "confiance": "elevee | moyenne | insuffisante",
  "mise_en_garde": "texte optionnel"
}

```

L'étape 7 est la garantie centrale du produit. Le modèle ne peut citer que des articles présents dans le contexte qu'on lui a fourni ; toute citation d'un identifiant absent fait rejeter la réponse entière. C'est mécanique, pas déclaratif — et c'est ce qui rend le critère « zéro réponse inventée » réellement vérifiable.

4. Schéma de base de données

PostgreSQL 16 + extension pgvector. Une seule base pour le corpus et l'applicatif.

```

CREATE EXTENSION IF NOT EXISTS vector;

CREATE TABLE texte (
  id SERIAL PRIMARY KEY,
  sigle VARCHAR(20) NOT NULL,          -- AUSCGIE, CGI, CIMA...
  titre TEXT NOT NULL,
  type VARCHAR(30) NOT NULL,          -- acte_uniforme | code
  version VARCHAR(50) NOT NULL,       -- "LF 2026"
  date_consolidation DATE NOT NULL,
  source_url TEXT, source_sha256 CHAR(64), valide_par VARCHAR(120)
);

CREATE TABLE article (
  id SERIAL PRIMARY KEY,
  texte_id INT NOT NULL REFERENCES texte(id),
  numero VARCHAR(30) NOT NULL,        -- "92", "18 bis"
  chemin TEXT NOT NULL,               -- "Livre I > Titre II > Chap. 3"
  contenu TEXT NOT NULL,
  date_entree_vigueur DATE NOT NULL,
  date_abrogation DATE,               -- NULL = en vigueur
  embedding VECTOR(1536),
  recherche_fts TSVECTOR
);

CREATE INDEX idx_article_vec ON article
  USING hns w (embedding vector_cosine_ops);
CREATE INDEX idx_article_fts ON article USING gin (recherche_fts);
CREATE INDEX idx_article_actif ON article (texte_id)
  WHERE date_abrogation IS NULL;

CREATE TABLE utilisateur (
  id SERIAL PRIMARY KEY, email CITEXT UNIQUE NOT NULL,
  mot_de_passe_hash TEXT NOT NULL, plan VARCHAR(20) DEFAULT 'gratuit',
  quota_restant INT DEFAULT 5, quota_reinit_le DATE
);

CREATE TABLE conversation (
  id SERIAL PRIMARY KEY, utilisateur_id INT REFERENCES utilisateur(id),
  titre TEXT, cree_le TIMESTAMPTZ DEFAULT now()
);

CREATE TABLE message (
  id SERIAL PRIMARY KEY, conversation_id INT REFERENCES conversation(id),
  role VARCHAR(10) NOT NULL, contenu TEXT NOT NULL,
  cree_le TIMESTAMPTZ DEFAULT now()
);

CREATE TABLE citation (
  message_id INT REFERENCES message(id),
  article_id INT REFERENCES article(id),
  extrait TEXT, PRIMARY KEY (message_id, article_id)
);

```

Règle absolue : aucun UPDATE sur le contenu d'un article. Une modification légale clôture l'ancienne ligne (date_abrogation) et en insère une nouvelle. C'est ce qui rend l'historique consultable et les retours arrière possibles.

5. Contrats d'API

Points d'entrée REST exposés par le backend.

Méthode & route	Rôle	Réponse
POST /auth/inscription	Création de compte	201 + JWT
POST /auth/connexion	Authentification	200 + JWT
POST /chat/question	Cœur du produit : question → réponse sourcée. Corps : {question, conversation_id?}	200 réponse + citations · 402 quota épuisé · 422 hors corpus

Méthode & route	Rôle	Réponse
GET /conversations	Liste des conversations de l'utilisateur	200 liste
GET /conversations/{id}	Détail avec messages et citations	200 objet
GET /textes	Corpus disponible, versions et dates de consolidation	200 liste
GET /textes/{id}/sommaire	Arborescence du texte	200 arbre
GET /articles/{id}	Article complet, chemin, voisins précédent/suivant	200 objet
GET /recherche?q=	Recherche plein texte classique, sans LLM (gratuite, hors quota)	200 résultats
GET /moi/quota	Quota restant et date de réinitialisation	200 objet

6. Architecture Angular

Organisation du frontend en composants autonomes (standalone).

```

src/app/
  core/
    services/      api.service.ts . auth.service.ts . chat.service.ts
                  corpus.service.ts . quota.service.ts
    interceptors/  jwt.interceptor.ts . erreur.interceptor.ts
    guards/        auth.guard.ts
    models/        article.model.ts . message.model.ts . citation.model.ts
  fonctionnalites/
    accueil/       accueil.page.ts
    chat/          chat.page.ts
                  composants/ bulle-message . bloc-base-legale
                           champ-question . indicateur-quota
    bibliotheque/  bibliotheque.page.ts . sommaire-arbre
    article/       article.page.ts . fil-ariane
    historique/    historique.page.ts
    methodologie/  methodologie.page.ts
    partage/       composants/ bouton . carte . avertissement-deonto
    app.routes.ts  (lazy loading par fonctionnalite)

```

Choix structurants

- **État applicatif par signals** (Angular 18) plutôt qu'une librairie externe : le périmètre ne justifie pas NgRx.
- **Un service = une responsabilité.** chatService gère le fil de conversation ; corpusService met en cache les textes et articles déjà lus (ils ne changent pas pendant une session).
- **Le composant** bloc-base-legale **est réutilisé partout** : c'est la signature visuelle du produit, il ne doit exister qu'une seule fois dans le code.
- **Réponse en streaming** : affichage progressif du texte, puis apparition des citations une fois la validation serveur terminée — jamais l'inverse, pour ne pas afficher une citation qui serait ensuite rejetée.
- **Service worker (@angular/pwa)** : cache des assets et des articles consultés ; le chat exige la connexion, la bibliothèque non.

7. Sécurité et confidentialité

Mesures inscrites dès la conception.

Domaine	Mesure
Clé LLM	Stockée en variable d'environnement serveur, jamais transmise au navigateur ; aucun appel direct du frontend vers le fournisseur
Mots de passe	Hachage bcrypt (coût 12) ; aucune récupération en clair possible
Sessions	JWT court (30 min) + jeton de rafraîchissement ; invalidation à la déconnexion
Secret professionnel	Contenu des questions non exploité au-delà de l'historique de l'utilisateur ; effacement à la demande ; clause de non-entraînement exigée du fournisseur LLM
Injection de prompt	La question utilisateur est passée en message séparé, jamais concaténée au prompt système ; les articles du contexte sont délimités explicitement
Abus / coût	Limitation de débit par IP et par compte ; quota décompté côté serveur uniquement ; refus avant appel LLM si le score de pertinence est insuffisant
Transport	HTTPS obligatoire, en-têtes de sécurité (HSTS, CSP), CORS restreint au domaine du frontend

8. Environnements et déploiement

Chaîne de livraison.

Environnement	Composition	Usage
Local	Docker Compose : PostgreSQL+pgvector, API FastAPI, Angular en mode dev	Développement quotidien et démo de secours
Préproduction	Branche formateur déployée automatiquement	Revue par le formateur
Production / démo	Frontend sur Vercel · API + PostgreSQL sur Railway ou Render	Lien public du Demo Day

Plan B démo : l'instance Docker Compose locale est maintenue opérationnelle en permanence. Si l'hébergement gratuit tombe le jour J, la démonstration se fait en local, sans dépendance réseau autre que l'appel LLM — et des captures d'écran couvrent même ce cas.

9. Stratégie de tests

Ce qui est testé, et pourquoi c'est là que se joue la crédibilité.

Niveau	Périmètre	Outil
Unitaire — backend	Fusion des scores de recherche, seuil de refus, validation des citations , décompte du quota	pytest
Unitaire — frontend	Services Angular, formatage des réponses, rendu du bloc base légale	Jasmine / Karma
Intégration	Parcours API complet question → réponse sur une base de test réduite	pytest + base éphémère
Évaluation métier	50 questions-réponses validées par un professionnel , dont 5 questions pièges hors corpus devant produire un refus. Taux de citations correctes mesuré et publié.	Script dédié, rejoué à chaque évolution du pipeline
Bout en bout	Parcours principal : poser une question, ouvrir une citation, revenir au chat	Playwright (si le temps le permet)

Le jeu d'évaluation métier n'est pas un test parmi d'autres : c'est le seul qui mesure la promesse du produit. Il doit être constitué **avant** l'optimisation du pipeline, sinon il sera inconsciemment écrit pour être réussi.

10. Séquence de développement recommandée

Ordre de construction, du plus risqué au plus confortable.

Ordre	Travail	Pourquoi à ce moment
1	Script d'ingestion sur un seul texte (AUSCGIE) + base + embeddings	Sans corpus propre, rien d'autre n'a de sens
2	Recherche hybride en ligne de commande, sans interface ni LLM	Vérifier que les bons articles remontent avant d'ajouter de la complexité
3	Jeu d'évaluation de 50 questions rédigé et figé	Constitué avant l'optimisation, jamais après
4	Endpoint /chat/question complet avec validation des citations	Le cœur du produit, testable sans interface
5	Frontend Angular : chat, bloc base légale, lecture d'article	L'interface habille une logique déjà prouvée
6	Bibliothèque, historique, comptes et quota	Confort, sans risque technique
7	PWA, export PDF, calculateurs, polissage	Finition — sacrificable si le temps manque

La logique de cet ordre : les trois premières étapes ne produisent aucun écran visible, ce qui est psychologiquement inconfortable — mais elles éliminent tout le risque technique du projet. Construire l'interface en premier donnerait l'illusion d'avancer tout en laissant intact le seul vrai danger : un moteur qui ne trouve pas les bons articles.